

How to Add Token Renewal (Refresh Token Flow)

Here's where and how you could implement token renewal:

1. First Modification: Login Route (/auth/login)

When a user logs in successfully, generate **both**:

- A short-lived **access token** (e.g., 15 minutes)
- A long-lived **refresh token** (e.g., 7 days)
(Stored securely in the database)

```
// Inside /auth/login route, after password verification:
if (passwordCorrect) {
  const { _id, email, name } = foundUser;
  const payload = { _id, email, name };

  // Access Token (short-lived)
  const authToken = jwt.sign(
    payload,
    process.env.TOKEN_SECRET,
    { algorithm: 'HS256', expiresIn: "15m" } // Short expiry
  );

  // Refresh Token (long-lived, stored in DB)
  const refreshToken = jwt.sign(
    { _id },
    process.env.REFRESH_TOKEN_SECRET, // Use a different secret!
    { expiresIn: "7d" }
  );

  // Save refreshToken to the user in database
  await User.findByIdAndUpdate(_id, { refreshToken });

  // Send both tokens to client
  res.status(200).json({ authToken, refreshToken });
}
```

2. New Route: Token Refresh (/auth/refresh)

Add a new endpoint to exchange a valid refresh token for a new access token:

```
// POST /auth/refresh - Renews the access token
router.post('/refresh', async (req, res) => {
  const { refreshToken } = req.body;

  if (!refreshToken) {
    return res.status(400).json({ message: "Refresh token required" });
  }
}
```

```

    }

    try {
      // Verify the refresh token
      const decoded = jwt.verify(refreshToken,
process.env.REFRESH_TOKEN_SECRET);

      // Check if the token exists in the database (optional but
recommended)
      const user = await User.findById(decoded._id);
      if (!user || user.refreshToken !== refreshToken) {
        return res.status(403).json({ message: "Invalid refresh token" });
      }

      // Generate a new access token
      const payload = { _id: user._id, email: user.email, name:
user.name };
      const newAuthToken = jwt.sign(
        payload,
        process.env.TOKEN_SECRET,
        { algorithm: 'HS256', expiresIn: "15m" }
      );

      // Optionally, issue a new refresh token (rotation)
      const newRefreshToken = jwt.sign(
        { _id: user._id },
        process.env.REFRESH_TOKEN_SECRET,
        { expiresIn: "7d" }
      );

      // Update the refreshToken in the database
      await User.findByIdAndUpdate(user._id, { refreshToken:
newRefreshToken });

      res.status(200).json({ authToken: newAuthToken, refreshToken:
newRefreshToken });
    } catch (err) {
      res.status(403).json({ message: "Invalid or expired refresh
token" });
    }
  });
}

```

3. Client-Side Flow

1. **Initial Login:** Client receives authToken (15m expiry) and refreshToken (7d expiry).
2. When authToken expires:
 1. Client sends refreshToken to /auth/refresh.
 2. Server validates it → returns a new authToken.
3. **If refreshToken expires:** User must log in again.

Key Security Notes

1. Refresh Token Storage:
 - Always store refresh tokens in your database (User model) to allow revocation.
 - Never store them in `localStorage` (use `HttpOnly` cookies if possible).
2. Token Rotation:
 - Issue a new refresh token on each use (prevents token reuse if stolen).
3. Secrets:
 - Use different secrets for access tokens (`TOKEN_SECRET`) and refresh tokens (`REFRESH_TOKEN_SECRET`).

By Miguel Garrido Iglesias
miguel.garrido@ironhack.com | www.miguel.nz